

2do Parcial 1era Fecha 2024

Ejercicio 2

a) i)

```
1  class Afiliado {
2      private String nyap;
3      private Int edad;
4      private String domicilio;
5      private float[] credenciales;
6
7      public void setEdad(int edad){
8          this.edad=edad;
9      }
10     public int getEdad(){
11         return this.edad;
12     }
13     public float getCredencial(anio){
14         return this.credenciales[anio];
15     }
16 }
```

Aca podemos observar un Producto cartesiano en la clase Afiliado ya que podemos observar un conjunto de elementos heterogeneos ordenados en n-tuplas. Tambien podemos observar una correspondencia finita ya que en credenciales vemos un conjunto de indices asociado a un conjunto de elementos (float)

a) ii)

```
1  piezas = ("tuerca", "tornillo", "clavo")
2  herramientas = [(1, 'tuerca'), (2, 'tornillo'), (3, 'clavo'),
3                  (4, 'martillo'), (5, 'destornillador')]
```

Aca podemos ver un producto cartesiano de un conjunto de elementos ordenados en una n-tupla, y tambien podemos ver una correspondencia finita de productos cartesianos, donde cada elemento de la correspondencia finita es un conjunto de elementos heterogeneos ordenados en n-tuplas, al cual a cada uno le corresponde un indice

b)

Los chequeos dinamicos siempre se aseguran que el lenguaje sea debilmente tipado. **Falso**

Las tuplas en python y las listas de pascal son ejemplos de correspondencia finita. Falso

Los lenguajes funcionales, al igual que los lógicos, son débilmente tipados. Falso

Justificaciones

No hay una relación entre que un lenguaje sea chequeado dinámicamente y lo estricto que es con el tipado. Uno habla de cuando se hace el chequeo y otro de que tan estricto es

Las tuplas de python son productos cartesianos ya que son una n-tupla de elementos heterogéneos, mientras que las listas de pascal tienen un constructor recursivo

No existe una relación clara entre que tan estricto es un lenguaje con sus chequeos de tipos y su paradigma

Ejercicio 3

```
1  try{
2      for (int i = 1; i < 5; i++) {
3          if(i == 1){
4              System.out.println(Integer.toString(i));
5              rethrow("Uno");
6          }
7          else{
8              if(i == 2 || i == 3){
9                  switch(i){
10                     case 2:
11                         System.out.println(Integer.toString(i));
12                         rethrow("Dos");
13                         break;
14                     case 3:
15                         System.out.println(Integer.toString(i));
16                         rethrow("Tres");
17                         break;
18                 }
19             }
20             else{
21                 if(i == 4){
22                     System.out.println(Integer.toString(i));
23                     rethrow("Cuatro");
24                 }
25             }
26         }
27     }
28 }
29 catch(ThirdException | FourthException e){
30     System.out.println(e.getMessage());
31 }
32
```

```

33
34 void rethrow(String s) throws ThirdException, FourthException {
35     try {
36         try {
37             switch(s){
38                 case "Uno":
39                     throw new FirstException("Excepción 1");
40                     break;
41                 case "Dos":
42                     throw new SecondException("Excepción 2");
43                     break;
44                 case "Tres":
45                     throw new ThirdException("Excepción 3");
46                     break;
47                 case "Cuatro":
48                     throw new FourthException("Excepción 4");
49                     break;
50             }
51         } catch (SecondException e) {
52             ThirdException e1 = new ThirdException("Excepción 3");
53             throw e1;
54         }
55     } catch (FirstException e) {
56         FourthException e1 = new FourthException("Excepción 4");
57         throw e1;
58     }

```

a) Imprime "1", despues "Excepcion 4" y despues termina

b) No pasaria ya que estamos lanzando una excepcion, ese break de abajo es codigo inalcanzable, pero en el caso de que estemos hablando de una sentencia normal se aplicaria el falling through y se ejecutaria el case 2 tambien

Ejercicio 4

a. Tanto Java como C exigen que los switch tengan una clausula default. **Falso**

b. Los hechos en un lenguaje logico permiten definir relaciones entre objetos del lenguaje.

Verdadero

c. El tipo recursion es facilmente implementable en Java. **Verdadero**

d. En PL/1 si se genera una excepcion, se ejecuta el manejador correspondiente y el programa termina automaticamente. **Falso**

e. La sentencia else de python en el manejo de excepciones se ejecuta siempre sin importar si hubo ejecucion de manejador o no. **Falso**

Justificaciones

- a.** La clausula default no es necesaria, si no esta y un valor no coincide con ninguna rama del condicional, el switch no ejecuta nada y sigue la ejecucion normal con lo que sigue abajo
- b.** Un lenguaje logico permite definir relaciones entre objetos como por ejemplo `tiene(coche, ruedas)` o `longitud([], 0)`
- c.** Se puede implementar facilmente, por ejemplo un arbol se suele implementar teniendo nodos dentro del mismo nodo, los hijos
- d.** En PL/1, si se genera una excepcion, se maneja con el manejador correspondiente y se reanuda la ejecucion en la instruccion siguiente a la que provoco el fallo
- e.** La sentencia else de python se ejecuta si no se produjo ninguna excepcion dentro del try